

STATE MANAGEMENT

Technical Documentation

Provider (classic) · Riverpod Todo App

Project: **advanced_firebase**

Field	Value
Primary branch (this guide)	todo_project
Bound branch (Provider)	advanced_state_manegment
Related branch	riverpoodStateManegment
Packages	provider ^6.x · flutter_riverpod ^3.4.2
Audience	Engineers learning shared app state in Flutter
Tone	Technical — architecture, ownership, APIs

This guide binds two learning branches into one terminology model: how we moved Firebase auth onto classic **Provider**, then how the **Todo** feature on **todo_project** owns list state with **Riverpod**.

1. Table of Contents

- 2. Purpose & Scope
- 3. Branch Map — How the Codebase Evolved
- 4. Unified Terminology Glossary
- 5. Core Principle — Single Source of Truth
- 6. Part A — Classic Provider (advanced_state_manegment)
 - 6.1 Why we left local setState
 - 6.2 Layered architecture
 - 6.3 MultiProvider wiring
 - 6.4 AuthProvider API
 - 6.5 UI consumption (watch / read)
- 7. Part B — Riverpod Todo (todo_project)
 - 7.1 Folder layout
 - 7.2 Domain model
 - 7.3 Notifier + Provider declaration
 - 7.4 ProviderScope entry point
 - 7.5 HomeScreen consumption
 - 7.6 Mutation flow (add / toggle / edit / delete)
- 8. Side-by-Side Comparison
- 9. Decision Guide — When to use which
- 10. Verification Checklists
- 11. Appendix — File Index

2. Purpose & Scope

This document is a technical walkthrough of **shared state ownership** in the **advanced_firebase** Flutter project. It does not re-teach Flutter widgets. It explains *where state lives*, *who mutates it*, and *how the UI subscribes* — first with the classic **provider** package on auth, then with **flutter_riverpod** on the Todo list.

The two branches are intentionally bound together. They solve the same class of problem (stale UI, duplicated services, unclear ownership) with different libraries. Using one glossary across both avoids the common confusion of treating “Provider” as a single product when it is both a package name and a Riverpod concept.

Scope note

Firestore_project currently focuses on Riverpod Todo (in-memory list). advanced_state_manegment focuses on Firebase Auth via ChangeNotifier. Firebase Auth screens are not the subject of the Todo branch; they are documented here so the Provider pattern remains part of the same learning path.

3. Branch Map — How the Codebase Evolved

Treat the repository as a learning timeline, not three unrelated apps:

```

master / early commits
  ■ Firebase setup, Auth UI, Firestore, FCM, SharedPreferences
  ▼
advanced_state_manegment
  ■ Migrated auth from local setState / StreamBuilder → classic Provider
  ■ MultiProvider + AuthProvider (ChangeNotifier) + AuthService
  ▼
riverpoodStateManegment
  ■ Introduced Riverpod (numbers demo / shared list across screens)
  ▼
todo_project    ← YOU ARE HERE
  Riverpod Todo: model + StateNotifier + todoProvider + HomeScreen
  
```

Figure 1 — Branch progression from Firebase baseline → Provider auth → Riverpod Todo

Branch	State library	Domain focus	Root wrapper
advanced_state_manegment	provider (classic)	Firebase Auth session	MultiProvider
riverpoodStateManegment	flutter_riverpod	Shared numbers list	ProviderScope
todo_project	flutter_riverpod	Todo CRUD list	ProviderScope

4. Unified Terminology Glossary

Use these terms consistently when discussing either branch. Words in the left column are the **canonical terms** for this project.

Term	Meaning in this project
State	Data that can change over time and that the UI must reflect (user session, todo list, loading flags).
Single source of truth	Exactly one owner for a piece of state. Screens listen; they do not each keep a private copy.
Service	Thin I/O wrapper (Firebase Auth, Firestore, prefs). No UI rebuild responsibility.
State holder	Object that owns mutable state and notifies listeners (ChangeNotifier or Riverpod Notifier / StateNotifier).
Provider (package)	pub.dev package provider — used on advanced_state_manegment.
Provider (Riverpod)	A named declaration that exposes state or a notifier to the widget tree via ref.
ChangeNotifier	Flutter class that calls notifyListeners() after mutation. Used by AuthProvider.
Notifier / StateNotifier	Riverpod types that hold state and expose methods to mutate it.
Watch	Subscribe to state and rebuild when it changes. Provider: context.watch. Riverpod: ref.watch.
Read	One-shot access for events/methods — do not subscribe. Provider: context.read. Riverpod: ref.read.
Root scope	App-wide container. Provider: MultiProvider. Riverpod: ProviderScope.
Consumer widget	Widget that can access the DI/state container. Riverpod: ConsumerWidget / ConsumerStatefulWidget.
Dependency injection (DI)	Creating services/providers once at the root and injecting them downward.
Immutability (lists)	Prefer state = [...state, item] over mutating a list in place without reassignment.

Naming trap

Do not say “we use Provider” without specifying the package. Say “classic Provider (ChangeNotifier)” or “Riverpod provider (todoProvider)”. Firebase Auth also exports a type named AuthProvider — on the auth branch we hide that import when needed.

5. Core Principle — Single Source of Truth

Both migrations answer the same failure mode: **data is correct in memory, but the UI is wrong**. That happens when multiple widgets own or mutate the same conceptual state with local setState, or when a child mutates a shared list reference and the parent never rebuilds.

5.1 The pattern (library-agnostic)

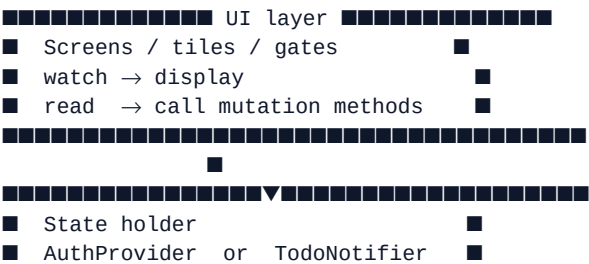




Figure 2 — Layering shared by both branches

Rule of thumb: **UI listens; state holder owns; service talks to the outside world.** If a screen still creates `AuthService()` or keeps `List<Todo>` in its `State` class, ownership has leaked and you will reintroduce stale UI.

6. Part A — Classic Provider (advanced_state_manegment)

Branch **advanced_state_manegment** migrated Firebase authentication from per-screen `AuthService()` + `setState` / `StreamBuilder` to a shared **AuthProvider** backed by **ChangeNotifier**.

6.1 Why we left local `setState`

Problem before	Solution after
Every screen created its own <code>AuthService()</code>	One shared <code>AuthService</code> from <code>MultiProvider</code>
Loading / errors lived in each screen's <code>setState</code>	Loading / errors live in <code>AuthProvider</code>
<code>AuthGate</code> used <code>StreamBuilder</code> + local service	<code>AuthGate</code> watches <code>AuthProvider</code>
Auth state duplicated across widgets	Single source of truth: <code>_user</code> in <code>AuthProvider</code>

6.2 Layered architecture

```

UI (Login / Signup / Home / AuthGate)
  └─ context.watch / context.read
    └─ ▼
      AuthProvider (ChangeNotifier – user, loading, error)
        └─ calls
          └─ ▼
            AuthService (Firebase Auth only)
  
```

By design, `SharedPreferences`, FCM setup, and Firestore profile writes stay in the screens after a successful login/register. `AuthProvider` stays focused on **session state** only. That is intentional separation of concerns.

6.3 MultiProvider wiring

Entry point on the Provider branch (**lib/main.dart**):

```

return MultiProvider(
  providers: [
    Provider(create: (_) => AuthService()),
    ChangeNotifierProvider(
      create: (ctx) => AuthProvider(ctx.read<AuthService>()),
    ),
  ],
  child: MaterialApp(
    home: const AuthGate(),
  ),
);
  
```

Registration	Role
<code>Provider<AuthService></code>	Plain DI — one shared service instance; widgets do not “listen” to it.
<code>ChangeNotifierProvider<AuthProvider></code>	State holder — widgets rebuild on <code>notifyListeners()</code> .
<code>ctx.read<AuthService>()</code>	Constructor injection so <code>AuthProvider</code> and screens share the same service.

6.4 AuthProvider API

Path: **lib/providers/auth_provider.dart**. On construction it subscribes to `authStateChanges`. The first emission clears `isInitializing` so `AuthGate` can leave the spinner.

Member	Role
<code>user</code>	Current Firebase User?

Member	Role
isLoggedIn	user != null — AuthGate routing
isInitializing	true until first stream event
isLoading	login / register in progress
errorMessage	Last human-readable auth error
login(...)	Future<bool> — success/fail for post-login side effects
register(...)	Future<UserCredential?> — credential for Firestore create
logout()	Signs out; stream clears user
clearError()	Resets errorMessage
dispose()	Cancels stream subscription

Mutation pattern inside AuthProvider

```

_isLoading = true;
_errorMessage = null;
notifyListeners();           // UI shows spinner / disabled button

try {
  await _authService.login(...);
  return true;
} on FirebaseAuthException catch (e) {
  _errorMessage = _messageForLoginError(e);
  return false;
} finally {
  _isLoading = false;
  notifyListeners();         // UI clears spinner
}

```

6.5 UI consumption (watch / read)

API	When to use	Example
context.watch<T>()	In build — need rebuilds	AuthGate, Login loading flag
context.read<T>()	In callbacks — one-shot	auth.login(...), auth.logout()

AuthGate is the clearest example of watch-driven routing: initializing → spinner; isLoggedIn → HomeScreen; else → LoginScreen. LoginScreen watches isLoading for the button, and reads AuthProvider inside _login() to call the method.

7. Part B — Riverpod Todo (todo_project)

Branch **todo_project** is the current working branch. It applies the same ownership rules to a Todo list using **flutter_riverpod**. The list is the single source of truth; HomeScreen does not store `List<Todo>` in a `StatefulWidget`.

7.1 Folder layout

Path	Responsibility
lib/main.dart	ProviderScope + MaterialApp entry
lib/models/todo.dart	Domain entity (id, title, completion flag) + copyWith / JSON
lib/notifiers/todo_notifiers.dart	State holder — mutates <code>List<Todo></code>
lib/providers/todo_provider.dart	Riverpod declaration binding notifier ↔ UI
lib/screen/home_screen.dart	ConsumerWidget — watch list, read mutations
lib/services/todo_title.dart	TodoTile presentational widget
lib/widget/.../add_todo_dialog.dart	Dialog for creating a title

Layering note

On this branch there is no `Firebase/ToDoService` yet — the notifier is the store. That is fine for learning. When you add persistence, introduce a `ToDoService` and keep the notifier as the state holder that calls the service (same shape as `AuthProvider` → `AuthService`).

7.2 Domain model

The `Todo` entity is a plain Dart class. Prefer immutable updates via `copyWith` so the notifier can replace items in the list without mutating fields on the old instance.

```
class Todo {
  Todo({required this.id, required this.title, required this.isDone});

  final String id;
  final String title;
  final bool isDone;

  Todo copyWith({String? id, String? title, bool? isDone}) { ... }

  factory Todo.fromJson(Map<String, dynamic> json) => ...
  Map<String, dynamic> toJson() => ...
}
```

Technical recommendation: keep one field name for completion across model, notifier, and tile (e.g. consistently **isDone** or consistently **completed**). Mixed naming is a common source of analyzer errors when the branch is still WIP.

7.3 Notifier + Provider declaration

The notifier owns `List<Todo>`. Methods assign a **new** list to state so Riverpod can detect the change and notify watchers.

```
// Conceptual shape used by HomeScreen on todo_project:

class TodoNotifier extends StateNotifier<List<Todo>> {
  TodoNotifier() : super([]);

  void addTodo(String title) {
    final todo = Todo(
```



```

        id: DateTime.now().millisecondsSinceEpoch.toString(),
        title: title,
        isDone: false,
    );
    state = [...state, todo];          // immutable append
}

void removeTodo(String id) {
    state = state.where((t) => t.id != id).toList();
}

void toggleTodo(String id) {
    state = [
        for (final t in state)
            if (t.id == id) t.copyWith(isDone: !t.isDone) else t,
    ];
}

void updateTodo(String id, String title) {
    state = [
        for (final t in state)
            if (t.id == id) t.copyWith(title: title) else t,
    ];
}
}

final todoProvider =
    StateNotifierProvider<TodoNotifier, List<Todo>>(
    (ref) => TodoNotifier(),
    );

```

Symbol	Meaning
StateNotifier<List<Todo>>	Holds the list; expose methods; assign state to notify.
StateNotifierProvider	Riverpod provider that exposes both the list and .notifier
ref.watch(todoProvider)	Rebuild when the list changes
ref.read(todoProvider.notifier)	Call addTodo / removeTodo / toggle / update

7.4 ProviderScope entry point

```

void main() {
    // ProviderScope is required: it holds the Riverpod container.
    runApp(const ProviderScope(child: MyApp()));
}

```

Without `ProviderScope`, any `ref.watch` / `ref.read` call fails at runtime. This is the Riverpod equivalent of `MultiProvider` on the `auth` branch — the **root scope** that owns the container for the whole app.

7.5 HomeScreen consumption

HomeScreen is a **ConsumerWidget**. That gives `build(context, ref)`. It watches the list for rendering and reads the notifier for user actions.

```
class HomeScreen extends ConsumerWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final todos = ref.watch(todoProvider);

    return Scaffold(
      appBar: AppBar(title: const Text("Riverpod Todo")),
      body: todos.isEmpty
        ? const Center(child: Text("No Todos"))
        : ListView.builder(
            itemCount: todos.length,
            itemBuilder: (_, index) {
              final todo = todos[index];
              return TodoTile(
                todo: todo,
                onDelete: () =>
                  ref.read(todoProvider.notifier).removeTodo(todo.id),
                onToggle: () =>
                  ref.read(todoProvider.notifier).toggleTodo(todo.id),
                onEdit: () async { /* dialog → updateTodo */ },
              );
            },
          ),
      floatingActionButton: FloatingActionButton(
        onPressed: () async {
          final title = await showDialog<String>(...);
          if (title != null && title.trim().isNotEmpty) {
            ref.read(todoProvider.notifier).addTodo(title);
          }
        },
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

7.6 Mutation flow (add / toggle / edit / delete)

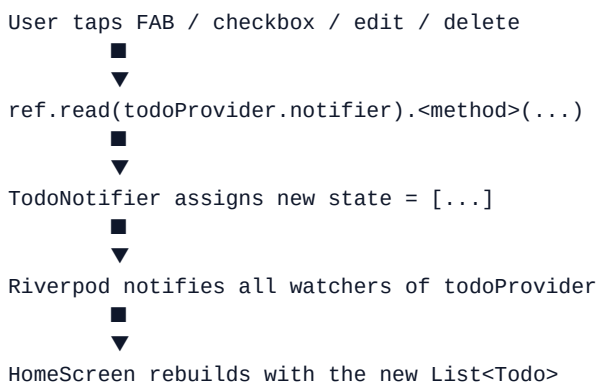


Figure 3 — End-to-end mutation on `todo_project`

Dialogs stay in the UI layer. They return a `String` (title). The notifier never imports `Flutter Material` — that keeps the state holder testable without a widget tree.

watch vs read (memorize)

Use	Where	Why
ref.watch	Inside build	Subscribe — rebuild when state changes
ref.read	onPressed / async callbacks	One-shot — call methods or peek once

Wrong: watch inside onPressed (confusing subscription). Wrong: read in build for values you display (UI will not update).

- Signup → Firebase user + Firestore profile.
- Logout → AuthGate returns to Login.

10.2 Riverpod Todo branch

- App starts under ProviderScope without runtime errors.
- Empty list shows “No Todos”.
- FAB → dialog → addTodo → tile appears.
- Checkbox → toggle → strikethrough / completion updates.
- Edit → dialog → updateTodo → title changes.
- Delete → removeTodo → tile disappears.
- No List<Todo> stored in HomeScreen State — only via ref.watch.

10.3 “Do I understand shared state?”

- I can explain single source of truth without naming a package.
- I know why root scope (MultiProvider / ProviderScope) is required.
- I use watch for display and read for events.
- I assign new state (or call notifyListeners) instead of mutating silently.
- I can map AuthProvider ↔ TodoNotifier using the comparison table.

11. Appendix — File Index

11.1 advanced_state_manegment (Provider)

File	Role
lib/main.dart	MultiProvider root
lib/providers/auth_provider.dart	ChangeNotifier auth state
lib/services/auth_service.dart	Firebase Auth wrapper
lib/widget/auth_gate.dart	Session routing via watch
lib/screen/login_screen.dart	read login / watch loading
lib/screen/signup_screen.dart	read register / watch loading
lib/screen/home_screen.dart	user + logout via Provider
docs/auth_provider_migration.md	Original Provider migration notes

11.2 todo_project (Riverpod)

File	Role
lib/main.dart	ProviderScope root
lib/models/todo.dart	Todo entity
lib/notifiers/todo_notifiers.dart	TodoNotifier state holder
lib/providers/todo_provider.dart	todoProvider declaration
lib/screen/home_screen.dart	ConsumerWidget UI
lib/services/todo_title.dart	TodoTile widget
docs/riverpod_basics.md	Earlier Riverpod numbers demo notes

Document status. Generated to reflect branch **todo_project** and to bind the Provider work on **advanced_state_manegment** under one terminology model. As the Todo notifier hardens (field naming, updateTodo, dialog implementation), keep Section 7 aligned with the source of truth in lib/.

— End of technical guide —